EX. NO:

DATE:

IMPLEMENT LINEAR REGRESSION USING SYNTHETIC DATASET

AIM:

To implement Linear Regression using a synthetic dataset by initializing model parameters, defining the linear model, loss function, training the model using gradient descent, and predicting continuous output values.

PROCEDURE:

1. **Generate Synthetic Dataset**
   Generate input features **X** using a normal distribution and compute output labels **Y** using the linear equation: Y=XW+bY = XW + bY=XW+b
2. **Initialize Model Parameters**
   Initialize weight vector **W** with small random values and bias **b** as zero.
3. **Define Linear Regression Model**
   The linear regression model is defined as:

   Y^=XW+b\hat{Y} = XW + bY^=XW+b

4. **Define Loss Function**
   Mean Squared Error (MSE) is used as the loss function:

L=1m∑i=1m(Y^i−Yi)2L = \frac{1}{m}\sum_{i=1}^{m} (\hat{Y}_i - Y_i)^2L=m1i=1∑m(Y^i −Yi)2

5. **Compute Gradients**
   Gradients of the loss with respect to model parameters are computed as:

∂L∂W=2mXT(Y^−Y)\frac{\partial L}{\partial W} = \frac{2}{m}X^T(\hat{Y} - Y)∂W∂L=m2 XT(Y^−Y) ∂L∂b=2m∑(Y^−Y)\frac{\partial L}{\partial b} = \frac{2}{m}\sum(\hat{Y} - Y)∂b∂L =m2∑(Y^−Y)

6. **Update Parameters (Gradient Descent)**
   Update the parameters using learning rate **α**:

W=W−α∂L∂WW = W - \alpha \frac{\partial L}{\partial W}W=W−α∂W∂L b=b−α∂L∂bb = b - \alpha \frac{\partial L}{\partial b}b=b−α∂b∂L

7. **Repeat Training**
   Repeat Steps 3 to 6 for a fixed number of epochs until the loss converges.
8. **Prediction**
   Predict output values for new inputs using:

Y^=XW+b\hat{Y} = XW + bY^=XW+b

9. **Visualization**
   Plot the synthetic dataset and regression line using Matplotlib and display the graph using plt.show().

ALGORITHM:

1)Input:
1.1 Feature matrix X of shape *(m, n)*
1.2 Target vector Y of shape *(m, 1)*
1.3 Learning rate α
1.4 Number of iterations T

2)Initialize:
2.1 Initialize weight vector W *(n, 1)* with small random values
2.2 Initialize bias b = 0

3) Training:
3.1 Compute prediction:

Y^=XW+b\hat{Y} = XW + bY^=XW+b

3.2 Compute loss using Mean Squared Error
3.3 Compute gradients
3.4 Update W and b

4)Repeat until convergence or T iterations.

5)Prediction:

Y^=XW+b\hat{Y} = XW + bY^=XW+b

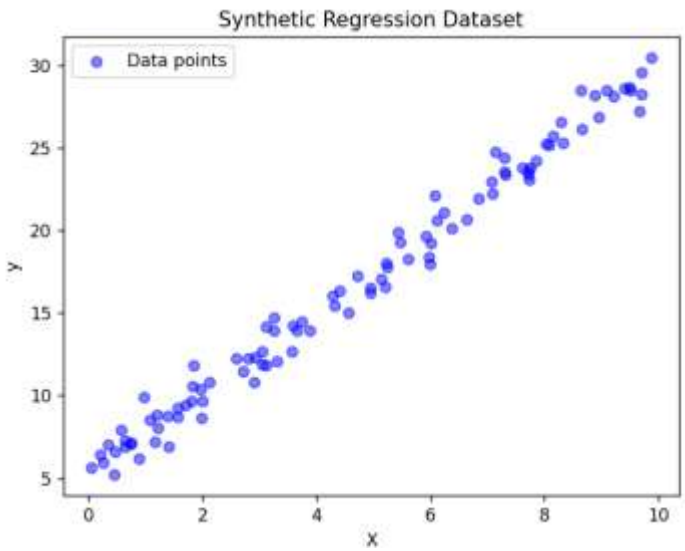
PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

n_samples = 100

true_slope = 2.5

true_bias = 5.0

X = np.random.rand(n_samples, 1) * 10

noise = np.random.randn(n_samples, 1)

y = true_slope * X + true_bias + noise

plt.scatter(X, y, color='blue', alpha=0.5, label="Data points")

plt.title("Synthetic Regression Dataset")

plt.xlabel("X")

plt.ylabel("y")

plt.legend()
```

```
plt.show()
w = np.random.randn(1)
b = np.random.randn(1)
print(f"Initial weight: {w}  Initial bias: {b}")
def predict(X, w, b):
    return X * w + b
def compute_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)
def gradient_descent(X, y, w, b, lr):
    n = len(y)
    y_pred = predict(X, w, b)
    dw = -2 * np.sum(X * (y - y_pred)) / n
    db = -2 * np.sum(y - y_pred) / n
    w = w - lr * dw
    b = b - lr * db
    return w, b
lr = 0.01
n_epochs = 100
loss_history = []
for epoch in range(n_epochs):
    y_pred = predict(X, w, b)
    loss = compute_loss(y, y_pred)
    loss_history.append(loss)
    w, b = gradient_descent(X, y, w, b, lr)
    if epoch % 10 == 0:
        print(f"Epoch {epoch}  Loss {loss:.4f}")
plt.plot(range(1, n_epochs + 1), loss_history, label="loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.title("Training Loss over Epochs")
plt.legend()
plt.show()
```

```
X_test = np.array([[0], [2], [4], [6], [8], [10]])
y_test_pred = predict(X_test, w, b)
plt.scatter(X, y, color='blue', alpha=0.5, label="Training Data")
plt.plot(X_test, y_test_pred, color='red', label="Model Prediction")
plt.title("Model Predictions on test data")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
print(f"Final Weight: {w}  Final bias: {b}")
```

SAMPLE OUTPUT:



Initial weight: [0.01300189] Initial bias: [1.45353408]

Epoch 0 Loss 285.2328

Epoch 10 Loss 3.5865

Epoch 20 Loss 3.2938

Epoch 30 Loss 3.0320

Epoch 40 Loss 2.7977

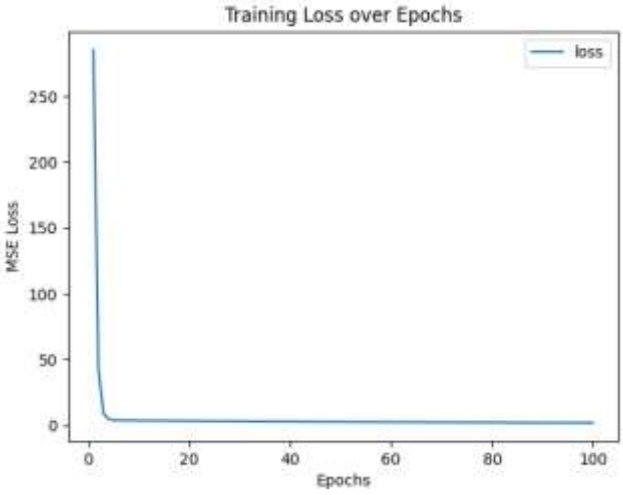
Epoch 50 Loss 2.5881

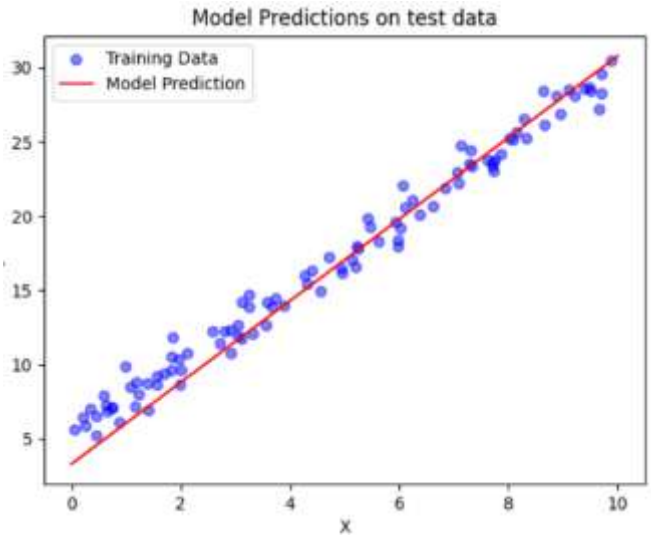
Epoch 60 Loss 2.4005

Epoch 70 Loss 2.2327

Epoch 80 Loss 2.0826

Epoch 90 Loss 1.9482





Final Weight: [2.74556366] Final bias: [3.31825682]

RESULT:

Thus, the Python program to implement SoftMax Regression using the Iris dataset is executed successfully.

EX. NO:

DATE:

IMPLEMENT SOFTMAX REGRESSION USING IRIS DATASET

AIM:

To implement the python program for softmax regression by initializing model parameters, defining the model, loss function, classification accuracy, followed by model training and prediction.

PROCEDURE:

1. Load the dataset.
2. Preprocess the data.
   2.1. Apply one-hot encoding for labels to convert it into a nominal data.
   2.2. Standard normalization is applied for features.
3. Initialize model parameters (weights, biases).
4. Define softmax function, i.e.,

$$Y_i = \frac{e^{z_i}}{\sum_{i=1}^{n} e^{z_i}}$$

5. Define forward propagation.

$$A = WX + B$$
$$Z = \text{softmax}(A)$$

6. Calculate loss function.

$$L = \sum_{i=1}^{n} y_i \log y_i$$

7. Compute the accuracy.

$$\text{Accuracy} = \frac{\text{Total no. of correctly predicted samples}}{\text{Total no. of test samples}}$$
$$= \frac{TP + TN}{TP + TN + FP + FN}$$

8. Train the model by repeating the following operations for epochs:
   8.1. Forward propagation
   8.2. Compute Loss
   8.3. Gradient Descent

$$dW = -2x(y - \text{mean}(y))$$
$$dB = -2(y - \text{mean}(y))$$

   8.4. Update the parameters

$$W_{new} = W_{old} - (\text{learning rate} \times dW)$$
$$b_{new} = b_{old} - (\text{learning rate} \times dB)$$

9. Prediction with test data.

ALGORITHM:

1. Input:

1.1. Feature matrix X of shape (m, n), where *m* is the number of samples and *n* is the number of features.
1.2. Target labels Y (one-hot encoded if needed) with *k* classes.
1.3. Learning rate α (alpha).
1.4. Number of iterations T.

2. Initialize:

2.1. Initialize weight matrix **W** of shape (n, k) with small random values.
2.2. Initialize bias vector **b** of shape (1, k) with zeros.

3. For each iteration (Gradient Descent Loop):

3.1. Compute logits (raw scores before softmax).
3.2. Apply the softmax function to obtain class probabilities.
3.3. Compute the loss (Cross-Entropy Loss).
3.4. Compute gradients.
3.5. Update parameters.
4.Repeat until convergence or maximum iterations reached.

5. Prediction:

5.1. Compute logits:

$$Z = XW + b$$

5.2. Apply softmax:

$$P = \frac{e^Z}{\sum e^Z}$$

5.3. Choose class with highest probability.

PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder, StandardScaler

iris = datasets.load_iris()

X = iris.data  # Features: Sepal & Petal length/width

y = iris.target  # Class labels (0, 1, 2)

encoder = OneHotEncoder(sparse_output=False)

Y = encoder.fit_transform(y.reshape(-1, 1))

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42)
def initialize_parameters(n_features, n_classes):
    W = np.random.randn(n_features, n_classes) * 0.01
    b = np.zeros((1, n_classes))
    return W, b
def softmax(z):
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)
def forward_propagation(X, W, b):
    Z = np.dot(X, W) + b
    A = softmax(Z)
    return A
def compute_loss(Y, A):
    loss = -np.mean(np.sum(Y * np.log(A + 1e-9), axis=1))
    return loss
def compute_accuracy(Y, A):
    predictions = np.argmax(A, axis=1)
    labels = np.argmax(Y, axis=1)
    return np.mean(predictions == labels) * 100
def train_softmax(X, Y, n_classes, learning_rate=0.1, epochs=1000):
    n_samples, n_features = X.shape
    W, b = initialize_parameters(n_features, n_classes)
    losses = []
    for epoch in range(epochs):
        A = forward_propagation(X, W, b)
        loss = compute_loss(Y, A)
        losses.append(loss)
        dW = np.dot(X.T, (A - Y)) / n_samples
        db = np.sum(A - Y, axis=0, keepdims=True) / n_samples
        W -= learning_rate * dW
        b -= learning_rate * db
```

```
        if epoch % 100 == 0:
            acc = compute_accuracy(Y, A)
            print(f"Epoch {epoch}: Loss = {loss:.4f}, Accuracy = {acc:.2f}%")
    return W, b, losses
def predict(X, W, b):
    A = forward_propagation(X, W, b)
    return np.argmax(A, axis=1)
n_classes = 3
W, b, losses = train_softmax(X_train, Y_train, n_classes)
plt.plot(losses)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss Curve for Softmax Regression on Iris Dataset")
plt.show()
A_test = forward_propagation(X_test, W, b)
test_accuracy = compute_accuracy(Y_test, A_test)
print(f"Test Accuracy: {test_accuracy:.2f}%")
X_sample = np.array([[5.1, 3.5, 1.4, 0.2]])
X_sample = scaler.transform(X_sample)
prediction = predict(X_sample, W, b)
print(f"Predicted Class: {iris.target_names[prediction][0]}")
```

SAMPLE OUTPUT:

Epoch 0: Loss = 1.0876, Accuracy = 68.33%

Epoch 100: Loss = 0.3397, Accuracy = 90.00%

Epoch 200: Loss = 0.2702, Accuracy = 90.00%

Epoch 300: Loss = 0.2290, Accuracy = 93.33%

Epoch 400: Loss = 0.2009, Accuracy = 94.17%

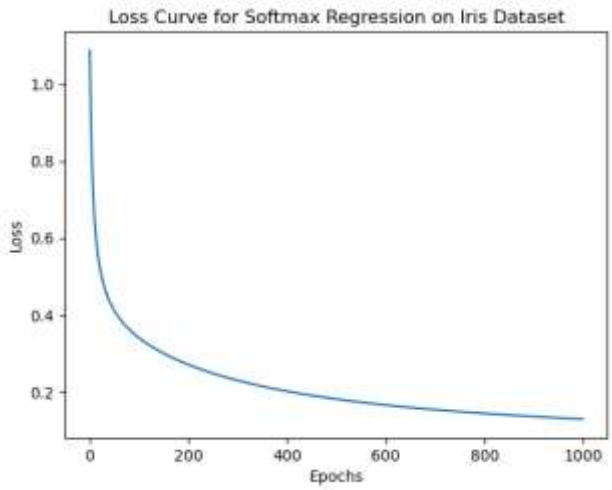
Epoch 500: Loss = 0.1806, Accuracy = 95.83%

Epoch 600: Loss = 0.1651, Accuracy = 96.67%

Epoch 700: Loss = 0.1530, Accuracy = 96.67%

Epoch 800: Loss = 0.1433, Accuracy = 96.67%

Epoch 900: Loss = 0.1353, Accuracy = 96.67%



Test Accuracy: 100.00%

RESULT:

Thus, the Python program to implement SoftMax Regression using the Iris dataset is executed successfully.

EX. NO:

DATE:

IMPLEMENT XOR PROBLEM USING MULTILAYER PERCEPTRON

AIM:

To implement the python program for XOR problem by initializing model parameters, defining the model, loss function, classification accuracy, followed by model training and prediction using Multi-Layer Perceptron.

PROCEDURE:

1. **Define XOR Dataset**
   Define the input matrix X and corresponding target output Y for the XOR problem.
2. **Initialize Network Parameters**
   Initialize the number of input, hidden, and output neurons.
   Randomly initialize weights W1,W2 and biases b1,b2 .
3. **Define Activation Function**
   Use the Sigmoid activation function and its derivative for non-linearity.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative: $\sigma'(x) = x(1 - x)$

4. **Perform Forward Propagation**
   Compute hidden layer output:

$$Z1 = XW1 + b1$$

$$A1 = \sigma(Z1)$$

Compute final output:

$$Z2 = A1W2 + b2$$

$$Y' = \sigma(Z2)$$

Compute Loss: $L = -[y\log(\hat{y}) + (1 - y)\log(1 - \hat{y})]$

5. **Perform Backpropagation**
   Compute output layer error gradient.

$$dW_2 = dZ_2 \cdot A_1^T$$
$$db_2 = dZ_2$$

6. **Update Weights and Biases**
   Update W1,W2,b1,b2 using gradient descent and learning rate.

$$W = W - \eta dW$$

$$b = b - \eta db$$

7. **Repeat Training**
   Repeat forward propagation, loss computation, backpropagation, and parameter update for a fixed number of epochs.
8. **Prediction**
   After training, compute final output and round the values to obtain XOR results.

ALGORITHM:

1) Input:

1.1 Feature matrix X (4 × 2)
1.2 Target vector Y (4 × 1)
1.3 Learning rate η
1.4 Number of epochs

2) Initialize:

2.1 Initialize W1, W2 randomly
2.2 Initialize b1, b2 randomly

3) Training:

3.1 Perform forward propagation
3.2 Compute loss using MSE
3.3 Perform backpropagation
3.4 Update weights and biases

4) Repeat until convergence or maximum epochs.

5) Prediction:

Compute final output and round values.

PROGRAM:

```
import numpy as np

def sigmoid(x):

    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):

    return x * (1 - x)


X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

y = np.array([[0], [1], [1], [0]])

np.random.seed(42)
```

```
input_neurons = 2
hidden_neurons = 2
output_neurons = 1

W1 = np.random.uniform(-1, 1, (input_neurons, hidden_neurons))
b1 = np.random.uniform(-1, 1, (1, hidden_neurons))
W2 = np.random.uniform(-1, 1, (hidden_neurons, output_neurons))
b2 = np.random.uniform(-1, 1, (1, output_neurons))

epochs = 10000
learning_rate = 0.1
loss_history = []
for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)
    error = y - final_output
    loss = np.mean(error ** 2)
    loss_history.append(loss)
    # Backpropagation
    d_output = error * sigmoid_derivative(final_output)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden_output)
    # Update weights and biases
    W2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
    if epoch % 1000 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.6f}")
```

```
# Testing the model
print("\nFinal Predictions:")
hidden_input = np.dot(X, W1) + b1
hidden_output = sigmoid(hidden_input)
final_input = np.dot(hidden_output, W2) + b2
final_output = sigmoid(final_input)
print(np.round(final_output))
```

SAMPLE OUTPUT:

```
Epoch 0, Loss: 0.254485
Epoch 1000, Loss: 0.250004
Epoch 2000, Loss: 0.249309
Epoch 3000, Loss: 0.242221
Epoch 4000, Loss: 0.173945
Epoch 5000, Loss: 0.048031
Epoch 6000, Loss: 0.016498
Epoch 7000, Loss: 0.008891
Epoch 8000, Loss: 0.005874
Epoch 9000, Loss: 0.004318
Final Predictions:
[[0.]
 [1.]
 [1.]
 [0.]]
```

RESULT:

Thus, the Python program to implement XOR problem using the Multi-Layer Perceptron has been executed successfully.

EX. NO:

DATE:

SPEECH RECOGNITION USING DEEP NEURAL NETWORK

AIM:

To design and implement a Deep Neural Network (DNN)-based speech classification system that extracts MFCC features from audio signals and accurately classifies spoken commands into predefined categories such as yes, no, up, down, and left.

PROCEDURE:

Step 1: Load Dataset

- Load speech dataset using TensorFlow Datasets.
- Extract audio signals and corresponding labels.

Step 2: Select Classes

- Choose specific classes:
  - yes, no, up, down, left
- Convert labels into numerical indices.

Step 3: Feature Extraction (MFCC)

- Convert audio signal into MFCC (Mel Frequency Cepstral Coefficients).

Important Formula:

1. **Fourier Transform:**

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$$

2. **Mel Scale Conversion:**

$$Mel(f) = 2595 \log_{10}(1 + \frac{f}{700})$$

3. **MFCC:**

$$MFCC = DCT(\log(|FFT(signal)|))$$

- Pad or truncate MFCC to fixed length (32).
- Flatten MFCC into a feature vector.

Step 4: Prepare Dataset

- Store extracted features in X
- Store labels in y
- Split data:
  - 80% training
  - 20% testing

Step 5: Build Neural Network

- Use Keras Sequential model:
  - Dense (512) + ReLU
  - Dropout (0.3)
  - Dense (256) + ReLU
  - Dropout (0.3)
  - Dense (128) + ReLU
  - Output layer (Softmax)

Step 6: Model Training

- **Loss function:**

$$Loss = -\sum y \log(\hat{y})$$

- Optimizer: Adam
- Train for 20 epochs

Step 7: Model Evaluation

- Evaluate using test data
- Calculate accuracy:

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$$

Step 8: Prediction

- Predict class probabilities
- Convert using:

Predicted Class=argmax(y)

Step 9: Performance Metrics

- Generate:
  - Classification Report (Precision, Recall, F1-score)
  - Confusion Matrix

Step 10: Visualization

- Plot confusion matrix using heatmap.

ALGORITHM:

1. Start
2. Load speech dataset
3. Select required classes
4. Initialize empty feature set X and labels y
5. For each audio sample:
   - Extract MFCC features
   - Pad/truncate to fixed size
   - Append to X
   - Append label to y
6. Convert X and y into arrays
7. Split dataset into training and testing sets
8. Build neural network model
9. Compile model with optimizer and loss function
10. Train model using training data
11. Save trained model
12. Evaluate model on test data
13. Predict output classes
14. Generate classification report
15. Plot confusion matrix

PROGRAM:

```
!pip install tensorflow==2.15 numpy==1.26.4 pandas librosa matplotlib seaborn scikit-learn
tensorflow-datasets pydub importlib_resources protobuf==3.20.3 pydub


import tensorflow as tf

import tensorflow_datasets as tfds

import numpy as np

import librosa

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report, confusion_matrix



dataset, info = tfds.load("speech_commands", split="train", with_info=True,as_supervised=True)

print(info.features["label"].names)



SELECTED_CLASSES = ["yes", "no", "up", "down", "left"]

class_indices = {label: i for i, label in enumerate(SELECTED_CLASSES)}

max_pad_len = 32


def extract_mfcc(audio, sr=16000, n_mfcc=13):

    audio = audio.astype(np.float32)

    mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=n_mfcc)

    pad_width = max_pad_len - mfccs.shape[1]

    if pad_width > 0:

        mfccs = np.pad(mfccs, pad_width=((0, 0), (0, pad_width)), mode='constant')

    else:

        mfccs = mfccs[:, :max_pad_len]

    return mfccs.flatten()
```

```
X, y = [], []
for audio, label in tfds.as_numpy(dataset.take(10000)):

    label_name = info.features["label"].int2str(label)
    if label_name in class_indices:
        features = extract_mfcc(audio)
        X.append(features)
        y.append(class_indices[label_name])

X = np.array(X)
y = np.array(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(512, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(len(SELECTED_CLASSES), activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test,
y_test))

model.save("speech_recognition_model.h5")
```

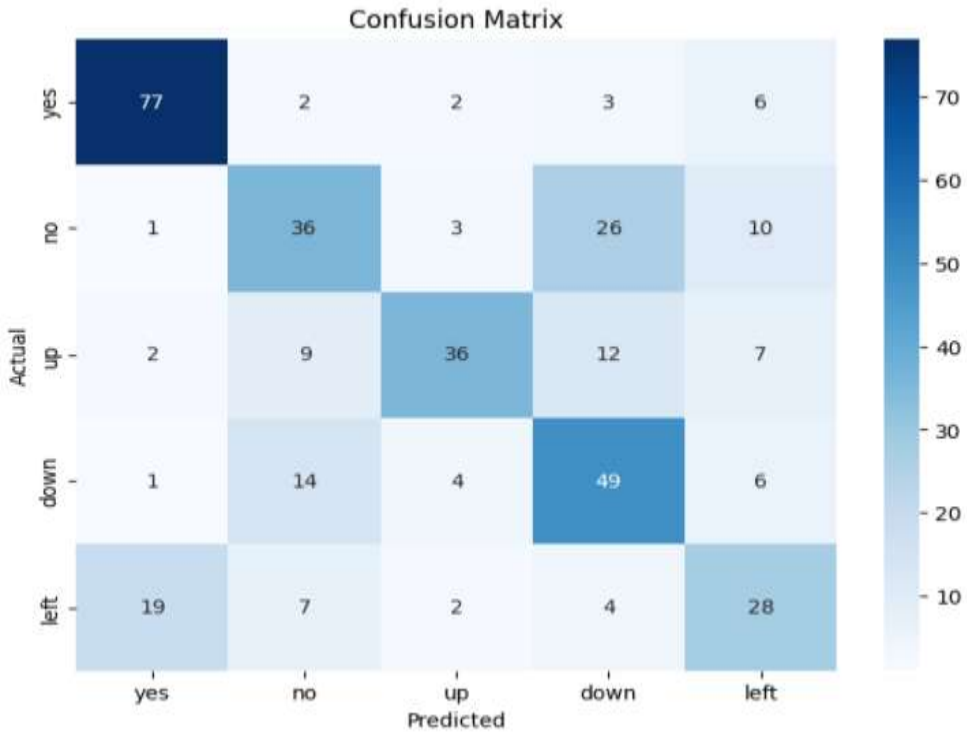
```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.2f}")

y_pred = model.predict(X_test)

y_pred_classes = np.argmax(y_pred, axis=1)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_classes, target_names=SELECTED_CLASSES))

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=SELECTED_CLASSES, yticklabels=SELECTED_CLASSES)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

SAMPLE OUTPUT:

```
['down', 'go', 'left', 'no', 'off', 'on', 'right', 'stop', 'up', 'yes', '_silence_', '_unknown_']
46/46 [==============================] - 0s 5ms/step - loss: 1.1900 - accuracy: 0.5318 -
val_loss: 1.0519 - val_accuracy: 0.5874
Epoch 20/20
46/46 [==============================] - 0s 5ms/step - loss: 1.2061 - accuracy: 0.5277 -
val_loss: 0.9836 - val_accuracy: 0.6175
12/12 [==============================] - 0s 2ms/step - loss: 0.9836 - accuracy: 0.6175
Test Accuracy: 0.62
```

Classification Report:

	precision	recall	f1-score	support
yes	0.77	0.86	0.81	90
no	0.53	0.47	0.50	76
up	0.77	0.55	0.64	66
down	0.52	0.66	0.58	74
left	0.49	0.47	0.48	60
accuracy			0.62	366
macro avg	0.62	0.60	0.60	366
weighted avg	0.62	0.62	0.61	366



RESULT:

Thus, the Python program to implement Speech recognition using the Deep Neural Network has been executed successfully.

EX. NO:

DATE:

IMPLEMENT HANDWRITTEN DIGIT RECOGNITION USING CONVOLUTION NEURAL NETWORK

AIM:

To implement a Python program for Handwritten Digit Recognition using Convolutional Neural Network (CNN) by loading the MNIST dataset, preprocessing the images, defining a CNN architecture using TensorFlow/Keras, training the model, evaluating its accuracy, and predicting digits from test images.

PROCEDURE:

1. Import Required Libraries

Import TensorFlow, Keras layers, NumPy, Matplotlib, and sklearn libraries for building the CNN model, visualization, and performance evaluation.

2. Load MNIST Dataset

Load the MNIST handwritten digit dataset using TensorFlow.

- Training images and labels: (x_train, y_train)
- Testing images and labels: (x_test, y_test)

Each image is 28 × 28 grayscale pixels representing digits 0–9.

3. Data Preprocessing

Normalize the pixel values to the range 0 to 1 by dividing by 255.

- Reshape the images to include a single channel dimension:

$$(28 \times 28) \rightarrow (28 \times 28 \times 1)$$

This format is required for CNN input.

4.Convolution Operation

Convolution extracts spatial features from the image.

Formula:

$$Z(i,j) = (X * K)(i,j) = \sum_m \sum_n X(i+m, j+n)K(m,n)$$

5.Activation Function (ReLU)

Apply the **Rectified Linear Unit (ReLU)** activation function.

Formula:

$$ReLU(x) = \max(0, x)$$

This introduces **non-linearity** and helps the network learn complex patterns.

6.Max Pooling

Pooling reduces spatial size and computation.

Formula:

$$P(i,j) = \max(Z_{region})$$

Where the **maximum value** from each pooling region is selected.

7.Flatten Layer & Dense Layer (Fully Connected Layer)

- Convert the 2D feature maps into a 1D feature vector.
- Compute weighted sum of inputs.

$$Z = WX + b$$

8.Softmax Activation Function

Softmax converts output values into **probability distribution**.

Formula:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

9.Loss Function

Formula: $L = -\log(p_{true})$

10.Optimization using Adam

Weights are updated using gradient descent.

General update rule:

$$W - W \quad \eta \frac{\partial L}{\partial W}$$

11.Prediction & Model Evaluation

$$Predicted\ Label = argmax(\hat{y})$$

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Predictions}$$

ALGORITHM:

1) Input

1.1 MNIST dataset containing handwritten digit images
1.2 Training images xtrainx_{train}xtrain and labels ytrainy_{train}ytrain
1.3 Testing images xtestx_{test}xtest and labels ytesty_{test}ytest
1.4 Number of epochs
1.5 CNN architecture parameters

2) Data Preprocessing

2.1 Normalize pixel values by dividing by 255
2.2 Reshape images to (28,28,1) format for CNN input

3) Model Initialization

3.1 Create Sequential CNN model
3.2 Add convolution and pooling layers
3.3 Add flatten and dense layers
3.4 Add output layer with Softmax activation

4) Model Compilation

4.1 Select Adam optimizer
4.2 Select Sparse Categorical Crossentropy loss function
4.3 Set Accuracy as evaluation metric

5) Training

5.1 Perform forward propagation
5.2 Compute classification loss
5.3 Perform backpropagation
5.4 Update weights using optimizer
5.5 Repeat for specified epochs

6) Model Evaluation

6.1 Test the trained CNN model using the test dataset
6.2 Compute test accuracy and test loss

7) Prediction

7.1 Select a test image
7.2 Reshape the image to CNN input format
7.3 Predict digit using the trained model
7.4 Display predicted digit

8) Performance Metrics

8.1 Generate Confusion Matrix
8.2 Compute Precision, Recall, F1-score
8.3 Display Classification Report

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import layers,models
import matplotlib.pyplot as plt
import numpy as np
from sklearn import metrics
from sklearn.metrics import classification_report


(x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
x_train,x_test=x_train/255.0,x_test/255.0
x_train=x_train.reshape(-1,28,28,1)
x_test=x_test.reshape(-1,28,28,1)


model=models.Sequential([
    layers.Conv2D(32 ,(3,3) ,activation ='relu',input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.Flatten(),
```

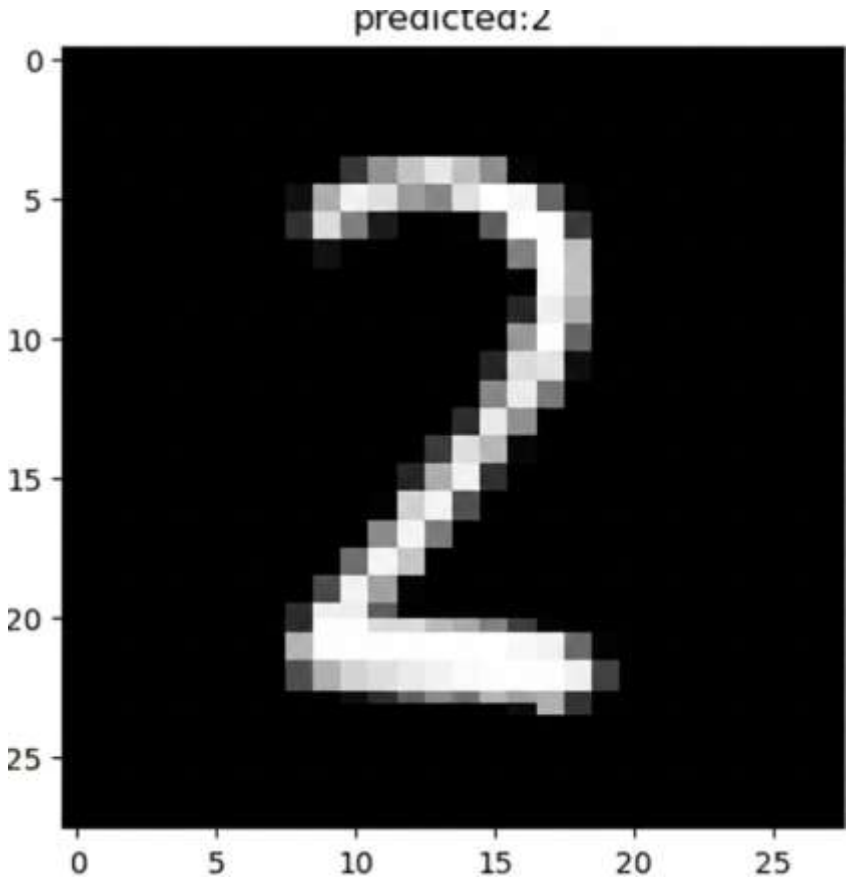
```
    layers.Dense(64,activation='relu'),
    layers.Dense(10,activation='softmax')

])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history=model.fit(x_train,y_train,epochs=5,validation_data=(x_test,y_test))

test_loss,test_acc=model.evaluate(x_test,y_test)
print("Test Accuracy :",test_acc)

def predict_digit(img):
    img=img.reshape(1,28,28,1)
    prediction=model.predict(img)
    return np.argmax(prediction)

sample_index=np.random.randint(len(x_test))
plt.imshow(x_test[sample_index].reshape(28,28),cmap='gray')
predicted_label=predict_digit(x_test[sample_index])
plt.title(f"predicted:{predicted_label}")
plt.show()

y_pred=np.argmax(model.predict(x_test),axis=1)
confusion_matrix=metrics.confusion_matrix(y_test,y_pred)
cm_display=metrics.ConfusionMatrixDisplay(confusion_matrix=confusion_matrix,display_label
s=list(range(10)))
cm_display.plot()
plt.show()

report=classification_report(y_test,y_pred)
print("Classification Report:")
print(report)
```

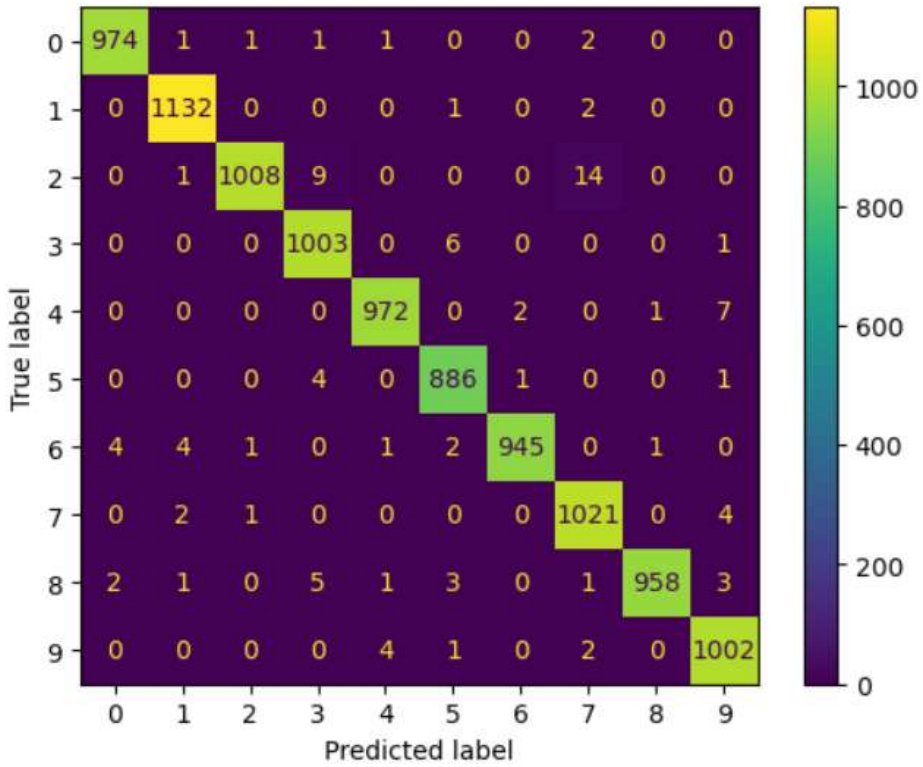
SAMPLE OUTPUT:

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy:
0.8807 - loss: 0.3645 - val_accuracy: 0.9866 - val_loss: 0.0467
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy:
0.9854 - loss: 0.0473 - val_accuracy: 0.9873 - val_loss: 0.0385
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy:
0.9893 - loss: 0.0330 - val_accuracy: 0.9905 - val_loss: 0.0283
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy:
0.9925 - loss: 0.0242 - val_accuracy: 0.9916 - val_loss: 0.0240
Epoch 5/5
```

1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **6s** 3ms/step - accuracy: 0.9948 - loss: 0.0174 - val_accuracy: 0.9901 - val_loss: 0.0319
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 1ms/step - accuracy: 0.9873 - loss: 0.0429
Test Accuracy : 0.9901000261306763
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 46ms/step

313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 942us/step





Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.99	1.00	0.99	1135
2	1.00	0.98	0.99	1032
3	0.98	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.98	0.99	0.99	1028
8	1.00	0.98	0.99	974
9	0.98	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

RESULT:

Thus, the Python program to implement Handwritten Digit Recognition using Convolutional Neural Network (CNN) has been executed successfully.

EX. NO:

DATE:

CONVOLUTION NEURAL NETWORK

AIM:

To implement the convolution operation on an input image using a kernel by considering different configurations such as multi-channel input, padding, stride, and valid convolution, and to generate the corresponding output feature map.

PROCEDURE:

Step 1: Define Input and Kernel

- Define the **input image** of size:H X W X C
  where C = number of channels.
- Define the **kernel/filter** of size:$F_h\ X\ F_w\ X\ C$

Step 2: Set Hyperparameters

- **Stride (S)** → step size of kernel movement
- **Padding (P)** → number of zero layers added around input

Step 3: Apply Padding (if required)

- If P>0, pad the input with zeros:
  New size=(H+2P)×(W+2P)

Step 4: Compute Output Dimensions

$$O_h = \frac{H + 2P - F_h}{S} + 1$$

$$O_w = \frac{W + 2P - F_w}{S} + 1$$

- If **no padding (valid convolution)**:
  O=H−F+1

Step 5: Initialize Output Feature Map

- Create an output matrix of size:

$$O_h \times O_w$$

Step 6: Perform Sliding Window Operation

- Move the kernel across the input using stride:

$$\text{start_i} = i \times S, \quad \text{start_j} = j \times S$$

Step 7: Extract Input Patch

- For each position (i,j)(i, j)(i,j), extract a patch:

$$\text{Patch} = F_h \times F_w$$

Step 8: Multi-Channel Convolution

- For each channel ccc:
  - Multiply input patch with corresponding kernel slice
  - Sum the results

$$\text{Output}(i,j) = \sum_{c=1}^{C} \sum (\text{Patch}_c \times \text{Kernel}_c)$$

Step 9: Element-wise Multiplication and Summation

$$\text{Output}(i,j) = \sum (\text{Patch} \odot \text{Kernel})$$

Step 10: Store Output

- Assign computed value to output feature map:

$$\text{Output}[i][j] = \text{computed sum}$$

Step 11: Repeat

- Repeat the process for all positions until full output is obtained.

1)2D CONVOLUTION WITH NO PADDING

ALGORITHM:

1) Start
2) Read input image and kernel.
3) Set padding = 0 and stride = 1.
4) Initialize output matrix.
5) Slide kernel only where it fully fits:

    a) Extract patch
    b) Multiply with kernel
    c) Sum values

6) Store result in output matrix.
7) Repeat for all valid positions.
8) Display output.
9) Stop

PROGRAM:

```
import numpy as np
# 1. Define a sample input image (e.g., a 5x5 grayscale image)
input_image = np.array([
    [1, 1, 1, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 0],
    [0, 1, 1, 0, 0]
], dtype=np.float32)
print("Input Image (5x5):")
print(input_image)
```

```
print("-" * 30)
# 2. Define a simple 3x3 kernel.
#This is a generic pattern-detection kernel.
kernel = np.array([
    [1, 0, 1],
    [0, 1, 0],
    [1, 0, 1]
], dtype=np.float32)

print("Kernel (3x3):")
print(kernel)
print("-" * 30)

# 3. Get the dimensions
i_h, i_w = input_image.shape
k_h, k_w = kernel.shape

# 4. Calculate the output dimensions for a 'valid' convolution (no padding, stride=1=> Move 1 step
each time)
#    The formula: output_size = input_size - kernel_size + 1
o_h = i_h - k_h + 1
o_w = i_w - k_w + 1

# 5. Initialize an empty output feature map
output_feature_map = np.zeros((o_h, o_w))

print(f"Input Dims: {i_h}x{i_w}, Kernel Dims: {k_h}x{k_w}, Output Dims: {o_h}x{o_w}")
print("-" * 30)

# 6. Perform the convolution: The Sliding Window
#    Outer loop: slide the kernel vertically (over rows)
for i in range(o_h):
    #    Inner loop: slide the kernel horizontally (over columns)
    for j in range(o_w):
        # Extract the current region of interest (patch) from the input image
        # This patch is the same size as the kernel (k_h x k_w)
        image_patch = input_image[i:i+k_h, j:j+k_w]

        # Store the result in the output feature map
        output_feature_map[i, j] = np.sum(image_patch * kernel)  # Element-wise multiplication and
sum

        # --- Detailed Print for the first few steps (optional) ---
        if i == 0 and j < 3: # Print for top-left and its neighbor
            print(f"Position (i={i}, j={j}):")
            print("Image Patch:")
            print(image_patch)
            print("Element-wise Product:")
            print(image_patch * kernel)
            print(f"Sum = {output_feature_map[i, j]}")
            print("---")
```

```
print("\nFinal Output Feature Map (after convolution):")
print(output_feature_map)
```

SAMPLE OUTPUT:

```
Input Image (5x5):
[[1. 1. 1. 0. 0.]
 [0. 1. 1. 1. 0.]
 [0. 0. 1. 1. 1.]
 [0. 0. 1. 1. 0.]
 [0. 1. 1. 0. 0.]]
------------------------------
Kernel (3x3):
[[1. 0. 1.]
 [0. 1. 0.]
 [1. 0. 1.]]
------------------------------
Input Dims: 5x5, Kernel Dims: 3x3, Output Dims: 3x3
------------------------------
Position (i=0, j=0):
Image Patch:
[[1. 1. 1.]
 [0. 1. 1.]
 [0. 0. 1.]]
Element-wise Product:
[[1. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]]
Sum = 4.0
---
Position (i=0, j=1):
Image Patch:
[[1. 1. 0.]
 [1. 1. 1.]
 [0. 1. 1.]]
Element-wise Product:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Sum = 3.0
---
Position (i=0, j=2):
Image Patch:
[[1. 0. 0.]
 [1. 1. 0.]
 [1. 1. 1.]]
Element-wise Product:
[[1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 1.]]
Sum = 4.0
```

Final Output Feature Map (after convolution):
[[4. 3. 4.]
 [2. 4. 3.]
 [2. 3. 4.]]

2)MULTI-CHANNEL CONVOLUTION

ALGORITHM:

1. **Start**
2. Read input image (with multiple channels) and kernel.
3. Set stride = 1 and no padding.
4. Initialize output matrix.
5. For each position in output:
    - Set sum = 0
    - For each channel:
        - Extract patch from that channel
        - Multiply with corresponding kernel slice
        - Add to sum
6. Store sum in output matrix.
7. Repeat for all positions.
8. Display output.
9. **Stop**

PROGRAM:

```
import numpy as np

# Given Parameters

input_size = 3

kernel_size = 2

padding = 0          # No padding

s = 1

input_channels = 2

number_of_filters = 1

# Define Input (3x3x2)

# Format: [row][column][channel]

input_image = np.array([

    [[1,9], [2,8], [3,7]],

    [[4,6], [5,5], [6,4]],

    [[7,3], [8,2], [9,1]]

])
```

```
print("Input Shape:", input_image.shape)
# Define Kernel (2x2x2)
kernel = np.array([
    [[1,0], [0,1]],
    [[0,1], [1,0]]
])
print("Kernel Shape:", kernel.shape)
# Get Dimensions
i_h, i_w, i_c = input_image.shape
k_h, k_w, k_c = kernel.shape
# Output Size Calculation
# Formula: (Input - Kernel) // Stride + 1
o_h = (i_h - k_h) // s + 1
o_w = (i_w - k_w) // s + 1
print("Output Size =", o_h, "x", o_w)
# Initialize Output
output = np.zeros((o_h, o_w))
# Convolution Operation
for i in range(o_h):
    for j in range(o_w):
        start_i = i * s
        start_j = j * s
        total_sum = 0
        for c in range(input_channels):
            # Extract patch
            patch = input_image[start_i:start_i + k_h,start_j:start_j + k_w, c]
            # Kernel slice
            kernel_slice = kernel[:, :, c]
            # Multiply and accumulate
            total_sum += np.sum(patch * kernel_slice)
        output[i][j] = total_sum
        print(f"Position (i={i}, j={j})")
```

```
        print("Start indices:", start_i, start_j)
        print("Output value:", total_sum)
        print("-" * 25)
# Final Output
print("\nFinal Output:")
print(output)
```

SAMPLE OUTPUT

```
Input Shape: (3, 3, 2)
Kernel Shape: (2, 2, 2)
Output Size = 2 x 2
Position (i=0, j=0)
Start indices: 0 0
Output value: 20
-------------------------
Position (i=0, j=1)
Start indices: 0 1
Output value: 20
-------------------------
Position (i=1, j=0)
Start indices: 1 0
Output value: 20
-------------------------
Position (i=1, j=1)
Start indices: 1 1
Output value: 20
-------------------------

Final Output:
[[20. 20.]
 [20. 20.]]
```

3)CONVOLUTION WITH PADDING:

ALGORITHM:

1. Start
2. Read input image and kernel.
3. Set padding = 1 and stride = 1.
4. Add zero padding around the input image.
5. Initialize output matrix.
6. Slide kernel over padded image:

    I. Extract patch
    II. Multiply with kernel
    III. Sum all values

7. Store result in output matrix.

8. Repeat for all positions.
9. Display output.
10. Stop

PROGRAM:

```
import numpy as np
input_image = np.array([
    [1, 1, 1, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 0],
    [0, 1, 1, 0, 0]
], dtype=np.float32)

print("Original Input Image (5x5):")
print(input_image)
kernel = np.array([
    [1, 0, 1],
    [0, 1, 0],
    [1, 0, 1]
], dtype=np.float32)

print("Kernel (3x3):")
print(kernel)
# 3. Set padding and stride
p = 1
s = 1
# 4. Add zero padding
padded_image = np.pad(input_image, pad_width=p, mode='constant', constant_values=0)
print("Padded Image (7x7):")
print(padded_image)
# 5. Get dimensions (after padding)
i_h, i_w = padded_image.shape
k_h, k_w = kernel.shape
# 6. Calculate output dimensions
# Formula: (n + 2p - k)/s + 1
o_h = (i_h - k_h) // s + 1 #  o_h = (i_h+ 2*p-k_h)//s + 1 if taken input image height instead of padded
image
o_w = (i_w - k_w) // s + 1 #  o_w = (i_w+ 2*p-k_w)//s + 1 if taken input image width instead of
padded image
print(f"Output Dimensions: {o_h} x {o_w}")
# 7. Initialize output feature map
output_feature_map = np.zeros((o_h, o_w))
# 8. Perform convolution
for i in range(o_h):
    for j in range(o_w):

        # Extract image patch
        image_patch = padded_image[i:i+k_h, j:j+k_w]
```

```
        # Element-wise multiplication and sum
        output_feature_map[i, j] = np.sum(image_patch * kernel)

        # Show first row calculations for clarity
        if i == 0:
            print(f"Position (i={i}, j={j})")
            print("Image Patch:")
            print(image_patch)
            print("Sum =", output_feature_map[i, j])
            print("---")

# 9. Final Output
print("\nFinal Output Feature Map:")
print(output_feature_map)
```

SAMPLE OUTPUT:

```
Original Input Image (5x5):
[[1. 1. 1. 0. 0.]
 [0. 1. 1. 1. 0.]
 [0. 0. 1. 1. 1.]
 [0. 0. 1. 1. 0.]
 [0. 1. 1. 0. 0.]]
Kernel (3x3):
[[1. 0. 1.]
 [0. 1. 0.]
 [1. 0. 1.]]
Padded Image (7x7):
[[0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 1. 1. 0. 0. 0.]
 [0. 0. 1. 1. 1. 0. 0.]
 [0. 0. 0. 1. 1. 1. 0.]
 [0. 0. 0. 1. 1. 0. 0.]
 [0. 0. 1. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]]
Output Dimensions: 5 x 5
Position (i=0, j=0)
Image Patch:
[[0. 0. 0.]
 [0. 1. 1.]
 [0. 0. 1.]]
Sum = 2.0
---
Position (i=0, j=1)
Image Patch:
[[0. 0. 0.]
 [1. 1. 1.]
 [0. 1. 1.]]
Sum = 2.0
---
Position (i=0, j=2)
Image Patch:
```

```
[[0. 0. 0.]
 [1. 1. 0.]
 [1. 1. 1.]]
Sum = 3.0
---
Position (i=0, j=3)
Image Patch:
[[0. 0. 0.]
 [1. 0. 0.]
 [1. 1. 0.]]
Sum = 1.0
---
Position (i=0, j=4)
Image Patch:
[[0. 0. 0.]
 [0. 0. 0.]
 [1. 0. 0.]]
Sum = 1.0
---

Final Output Feature Map:
[[2. 2. 3. 1. 1.]
 [1. 4. 3. 4. 1.]
 [1. 2. 4. 3. 3.]
 [1. 2. 3. 4. 1.]
 [0. 2. 2. 1. 1.]]
```

4)CONVOLUTION WITH STRIDE 2
ALGORITHM:

1. Start
2. Read input image and kernel.
3. Set padding = 1 and stride = 2.
4. Add zero padding to input image.
5. Initialize output matrix.
6. Move kernel with step size = 2:
7. Extract patch using stride
8. Multiply with kernel
9. Sum values
10. Store result in output.
11. Repeat until full image is covered.
12. Display output.
13. Stop

PROGRAM:

```
import numpy as np
input_image = np.array([
```

```
    [1, 1, 1, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 0],
    [0, 1, 1, 0, 0]
], dtype=np.float32)
print("Original Input Image (5x5):")
print(input_image)
print("-" * 30)
kernel = np.array([
    [1, 0, 1],
    [0, 1, 0],
    [1, 0, 1]
], dtype=np.float32)

print("Kernel (3x3):")
print(kernel)
print("-" * 30)

# 3. Set padding and stride
p = 1
s = 2   # stride changed to 2

# 4. Add zero padding
padded_image = np.pad(input_image, pad_width=p, mode='constant', constant_values=0)

print("Padded Image (7x7):")
print(padded_image)
print("-" * 30)

# 5. Get dimensions
i_h, i_w = padded_image.shape
k_h, k_w = kernel.shape

# 6. Calculate output dimensions
o_h = (i_h - k_h) // s + 1
o_w = (i_w - k_w) // s + 1

print(f"Output Dimensions: {o_h} x {o_w}")
print("-" * 30)

# 7. Initialize output feature map
output_feature_map = np.zeros((o_h, o_w))

# 8. Perform convolution
for i in range(o_h):
    for j in range(o_w):
        start_i = i * s
        start_j = j * s

        image_patch = padded_image[start_i:start_i+k_h,start_j:start_j+k_w]
```

```
        output_feature_map[i, j] = np.sum(image_patch * kernel)
        print(f"Position (i={i}, j={j})")
        print("Patch:")
        print(image_patch)
        print("Sum =", output_feature_map[i, j])
        print("---")

print("\nFinal Output Feature Map:")
print(output_feature_map)
```

SAMPLE OUTPUT:

```
Original Input Image (5x5):
[[1. 1. 1. 0. 0.]
 [0. 1. 1. 1. 0.]
 [0. 0. 1. 1. 1.]
 [0. 0. 1. 1. 0.]
 [0. 1. 1. 0. 0.]]
------------------------------
Kernel (3x3):
[[1. 0. 1.]
 [0. 1. 0.]
 [1. 0. 1.]]
------------------------------
Padded Image (7x7):
[[0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 1. 1. 0. 0. 0.]
 [0. 0. 1. 1. 1. 0. 0.]
 [0. 0. 0. 1. 1. 1. 0.]
 [0. 0. 0. 1. 1. 0. 0.]
 [0. 0. 1. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]]
------------------------------
Output Dimensions: 3 x 3
------------------------------
Position (i=0, j=0)
Patch:
[[0. 0. 0.]
 [0. 1. 1.]
 [0. 0. 1.]]
Sum = 2.0
---
Position (i=0, j=1)
Patch:
[[0. 0. 0.]
 [1. 1. 0.]
 [1. 1. 1.]]
Sum = 3.0
---
Position (i=1, j=0)
Patch:
```

```
[[0. 0. 1.]
 [0. 0. 0.]
 [0. 0. 0.]]
Sum = 1.0
---
Position (i=1, j=1)
Patch:
[[1. 1. 1.]
 [0. 1. 1.]
 [0. 1. 1.]]
Sum = 4.0
---
Position (i=1, j=2)
Patch:
[[1. 0. 0.]
 [1. 1. 0.]
 [1. 0. 0.]]
Sum = 3.0
---
Position (i=2, j=0)
Patch:
[[0. 0. 0.]
 [0. 0. 1.]
 [0. 0. 0.]]
Sum = 0.0
---
Position (i=2, j=1)
Patch:
[[0. 1. 1.]
 [1. 1. 0.]
 [0. 0. 0.]]
Sum = 2.0
---
Position (i=2, j=2)
Patch:
[[1. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Sum = 1.0
---

Final Output Feature Map:
[[2. 3. 1.]
 [1. 4. 3.]
 [0. 2. 1.]]
```

RESULT:

Thus, the convolution operation was successfully implemented on the input image using the given kernel by applying different configurations such as multi-channel processing, padding, stride, and valid convolution.

EX. NO:

DATE:

IMPLEMENT IMAGE-BASED OBJECT RECOGNITION USING GOOGLENET

AIM:

To implement a Python program for image-based object recognition using the GoogLeNet (InceptionV3) deep learning architecture by loading the CIFAR-10 dataset, preprocessing the images, applying transfer learning with a pre-trained InceptionV3 model, training the network, and evaluating the model performance using classification metrics.

PROCEDURE:

1. Import Required Libraries

Import TensorFlow, Keras layers, NumPy, Matplotlib, and Scikit-learn for building the model, visualization, and performance evaluation.

2Load CIFAR-10 Dataset

Load the CIFAR-10 dataset which contains 60,000 images belonging to 10 classes.

- Training images and labels: (x_train, y_train)
- Testing images and labels: (x_test, y_test)

3. Data Preprocessing

Normalize the pixel values to the range 0 to 1 by dividing by 255.

- Reshape the images to include a single channel dimension:

$$(28 \times 28) \rightarrow (28 \times 28 \times 1)$$

4.One-Hot Encoding of Labels

Convert class labels into categorical vectors.

General representation:

$$y_{encoded} = to_categorical(y)$$

5. Feature Extraction using GoogLeNet (InceptionV3)

Use InceptionV3 pre-trained on ImageNet for feature extraction.

Formula:

$$Z(i,j) = (X * K)(i,j) = \sum_m \sum_n X(i+m, j+n)K(m,n)$$

6.Activation Function (ReLU)

Apply the Rectified Linear Unit (ReLU) activation function.

Formula:

$$ReLU(x) = \max(0, x)$$

This introduces non-linearity and helps the network learn complex patterns.

6.Max Pooling

Pooling reduces spatial size and computation.

Formula:

$$P(i,j) = \max(Z_{region})$$

Where the maximum value from each pooling region is selected.

7.Flatten Layer & Dense Layer (Fully Connected Layer)

- Convert the 2D feature maps into a 1D feature vector.

- Compute weighted sum of inputs.

$$Z = WX + b$$

8.Softmax Activation Function

Softmax converts output values into **probability distribution**.

Formula:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

9.Loss Function

Formula: $L = -\log(p_{true})$

10.Optimization using Adam

Weights are updated using gradient descent.

General update rule:

$$W - W \quad \eta \frac{\partial L}{\partial W}$$

11.Prediction & Model Evaluation

$$Predicted\ Label = argmax(\hat{y})$$

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Predictions}$$

ALGORITHM:

1) Input

1.1 CIFAR-10 image dataset
1.2 Training images xtrainx_{train}xtrain and labels ytrainy_{train}ytrain
1.3 Testing images xtestx_{test}xtest and labels ytesty_{test}ytest
1.4 Learning rate
1.5 Number of epochs

2) Data Preprocessing

2.1 Normalize images by dividing pixel values by 255
2.2 Convert labels to one-hot encoded vectors
2.3 Resize images to 75 × 75 × 3

3) Model Initialization

3.1 Load InceptionV3 pre-trained model
3.2 Remove the top classification layer
3.3 Add Global Average Pooling layer
3.4 Add Dense layer with ReLU activation
3.5 Add final Softmax output layer

4) Freeze Base Model Layers

4.1 Set pre-trained layers as non-trainable
4.2 Train only newly added classification layers

5) Model Compilation

5.1 Use Adam optimizer
5.2 Use Categorical Crossentropy loss function
5.3 Use Accuracy as evaluation metric

6) Model Training

6.1 Perform forward propagation
6.2 Compute loss
6.3 Perform backpropagation
6.4 Update weights
6.5 Repeat for specified epochs

7) Prediction

7.1 Predict classes for test images
7.2 Convert probabilities to labels using argmax

8) Performance Evaluation

8.1 Generate Classification Report
8.2 Display Precision, Recall, and F1-Score

9) Visualization

9.1 Select a random test image
9.2 Display the image with true label and predicted label

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.layers import Dense,GlobalAveragePooling2D,Input
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import confusion_matrix,classification_report
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
(x_train,y_train),(x_test,y_test)=cifar10.load_data()

x_train,x_test=x_train/255.0,x_test/255.0
y_train=to_categorical(y_train,10)
y_test=to_categorical(y_test,10)
x_train=tf.image.resize(x_train,[75,75])
x_test=tf.image.resize(x_test,[75,75])

input_tensor=Input(shape=(75,75,3))
```

```
base_model=InceptionV3(weights='imagenet',include_top=False,input_tensor=input_tensor)
x=GlobalAveragePooling2D()(base_model.output)
x=Dense(256,activation='relu')(x)
predictions=Dense(10,activation='softmax')(x)
model=Model(inputs=base_model.input,outputs=predictions)


for layer in base_model.layers:
    layer.trainable=False

model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train,y_train,validation_data=(x_test,y_test),epochs=10,batch_size=64)

y_pred=model.predict(x_test)
y_pred_classes=np.argmax(y_pred,axis=1)
y_true_classes=np.argmax(y_test,axis=1)

class_report=classification_report(y_true_classes,y_pred_classes)
print("Classification Report:\n",class_report)

sample_index=np.random.randint(0,len(x_test))
sample_image=x_test[sample_index]
sample_true_label=np.argmax(y_test[sample_index])
sample_pred_label=np.argmax(y_pred[sample_index])
plt.imshow(sample_image)
plt.title(f"True Label:{sample_true_label},Predicted Label: {sample_pred_label}")
plt.axis('off')
plt.show()
```

SAMPLE OUTPUT:

```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44s 52ms/step - accuracy: 0.5243 - loss:
1.3809 - val_accuracy: 0.6172 - val_loss: 1.1202
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44s 56ms/step - accuracy: 0.6633 - loss:
0.9684 - val_accuracy: 0.6264 - val_loss: 1.0798
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44s 56ms/step - accuracy: 0.7066 - loss:
0.8364 - val_accuracy: 0.6388 - val_loss: 1.0806
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44s 57ms/step - accuracy: 0.7500 - loss:
0.7086 - val_accuracy: 0.6298 - val_loss: 1.1143
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44s 57ms/step - accuracy: 0.8024 - loss:
0.5723 - val_accuracy: 0.6256 - val_loss: 1.1869
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44s 57ms/step - accuracy: 0.8500 - loss:
0.4570 - val_accuracy: 0.6173 - val_loss: 1.2965
```

```
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45s 57ms/step - accuracy: 0.8933 - loss:
0.3464 - val_accuracy: 0.6160 - val_loss: 1.3910
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45s 57ms/step - accuracy: 0.9257 - loss:
0.2529 - val_accuracy: 0.6108 - val_loss: 1.4962
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45s 57ms/step - accuracy: 0.9497 - loss:
0.1860 - val_accuracy: 0.6025 - val_loss: 1.6205
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45s 57ms/step - accuracy: 0.9666 - loss:
0.1385 -
```

Classification Report:

	precision	recall	f1-score	support
0	0.60	0.72	0.65	1000
1	0.72	0.72	0.72	1000
2	0.52	0.51	0.51	1000
3	0.46	0.44	0.45	1000
4	0.54	0.51	0.52	1000
5	0.60	0.53	0.56	1000
6	0.58	0.68	0.62	1000
7	0.64	0.68	0.66	1000
8	0.75	0.67	0.71	1000
9	0.73	0.66	0.69	1000
accuracy			0.61	10000
macro avg	0.61	0.61	0.61	10000
weighted avg	0.61	0.61	0.61	10000

True Label:2,Predicted Label: 2

RESULT:

Thus, the Python program to implement image-based object recognition using the GoogLeNet has been executed successfully.

EX. NO:

DATE:

CATEGORIZE OPINIONS IN TEXT USING LONG SHORT TERM NETWORKS.

AIM:

To design and implement a system that categorizes text into positive or negative opinions using a Long Short-Term Memory (LSTM) neural network.

PROCEDURE:

1. Data Collection

A dataset of sentences is collected and labeled as:

Positive → 1

Negative → 0

2. Text Preprocessing

a) Tokenization

Each sentence is split into individual words.

b) Vocabulary Creation

A unique index is assigned to each word.

c) Encoding

Words are converted into numerical sequences using the vocabulary.

d) Padding

Sequences are padded with zeros to make all inputs of equal length.

3. Embedding Layer

Each word index is converted into a dense vector representation.

4. LSTM Computation

The LSTM processes the sequence using gating mechanisms:

Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Cell State Update

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Hidden State

$$h_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \cdot \tanh(C_t)$$

5. Fully Connected Layer

The final hidden state is passed through a linear layer to produce output.

6. Activation Function (Sigmoid)

$$\hat{y} = \frac{1}{1+e^{-y}}$$

7. Loss Function (Binary Cross Entropy)

$$L = -[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$$

8. Optimization

Model parameters are updated using backpropagation and an optimizer like Adam.

ALGORITHM:

1. Start
2. Input labeled text dataset
3. Tokenize sentences into words
4. Build vocabulary and assign indices
5. Encode sentences into numerical sequences
6. Apply padding to make equal length inputs
7. Initialize LSTM model parameters
8. Pass input through embedding layer
9. Feed embeddings into LSTM network
10. Extract last hidden state
11. Pass through fully connected layer
12. Apply sigmoid activation to get probability
13. Compute loss using Binary Cross Entropy
14. Perform backpropagation
15. Update weights using optimizer
16. Repeat steps 8–15 for multiple epochs
17. Test model on new sentence
18. If output > 0.5 → Positive, else Negative
19. Stop

PROGRAM:

```
import torch
import torch.nn as nn
import torch.optim as optim


sentences = [
    "this movie is amazing",
    "i hate this film",
    "the story is good",
    "very boring movie"
]

labels = torch.tensor([1,0,1,0])


vocab = {}
index = 0

for sentence in sentences:
    for word in sentence.split():
        if word not in vocab:
            vocab[word] = index
            index += 1


from torch.nn.utils.rnn import pad_sequence
```

```
encoded = [torch.tensor([vocab[w] for w in s.split()]) for s in sentences]

inputs = pad_sequence(encoded, batch_first=True)


class LSTMClassifier(nn.Module):

    def __init__(self, vocab_size, embed_size, hidden_size, output_size):
        super(LSTMClassifier, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_size)

        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)

        self.fc = nn.Linear(hidden_size, output_size)

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):

        x = self.embedding(x)

        lstm_out, _ = self.lstm(x)

        last_output = lstm_out[:, -1, :]

        out = self.fc(last_output)

        out = self.sigmoid(out)

        return out


vocab_size = len(vocab)

model = LSTMClassifier(
    vocab_size=vocab_size,
    embed_size=10,
    hidden_size=16,
    output_size=1
)


criterion = nn.BCELoss()

optimizer = optim.Adam(model.parameters(), lr=0.01)


for epoch in range(100):
```

```
    outputs = model(inputs)

    loss = criterion(outputs.squeeze(), labels.float())

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    if epoch % 10 == 0:
        print("Epoch:", epoch, "Loss:", loss.item())


test_sentence = "this movie is good"

test_encoded = torch.tensor([encode(test_sentence)])
prediction = model(test_encoded)
print(prediction)
```

SAMPLE OUTPUT:

```
Epoch: 0 Loss: 0.7013343572616577
Epoch: 10 Loss: 0.36982014775276184
Epoch: 20 Loss: 0.08687695860862732
Epoch: 30 Loss: 0.019808299839496613
Epoch: 40 Loss: 0.007909110747277737
Epoch: 50 Loss: 0.004645800217986107
Epoch: 60 Loss: 0.0033457002136856318
Epoch: 70 Loss: 0.0026775680016726255
Epoch: 80 Loss: 0.002259936649352312
Epoch: 90 Loss: 0.001961146015673876

tensor([[0.9993]], grad_fn=<SigmoidBackward0>)
```

RESULT:

Thus, the Python program to implement system that categorizes text into positive or negative opinions using a Long Short-Term Memory (LSTM) neural network has been executed successfully.

EX. NO:

DATE:

MACHINE TRANSLATION USING GATED RECURRENT UNITS

AIM:

To design and implement a machine translation using gated recurrent unit for sentence translation.

PROCEDURE:

1. Data Collection

A dataset of parallel sentences is prepared:

English sentence → Input

Hindi sentence → Target output

2. Text Preprocessing

a) Tokenization

Both input and target sentences are split into words.

b) Vocabulary Creation

Separate vocabularies are created for source (English) and target (Hindi).

c) Encoding

Words are converted into numerical indices.

d) Padding

Sequences are padded to ensure equal length.

3. Embedding Layer

Each word index is mapped to a dense vector representation.

4. Encoder (GRU)

The encoder processes the input sequence and produces a context vector:

$$h_t = \mathrm{GRU}(x_t, h_{t-1})$$

Final hidden state represents the sentence:

$$h_n = \text{Context Vector}$$

5. Decoder (GRU)

The decoder generates the translated sequence using the context vector:

$$s_t = \mathrm{GRU}(y_{t-1}, s_{t-1})$$

6.Output

$$y_t = W \cdot s_t + b$$

7. Activation Function (Sigmoid)

$$P(y_t) = \frac{e^{y_t}}{\sum e^{y_i}}$$

8. Loss Function (Binary Cross Entropy)

$$L = -\sum y \log(\hat{y})$$

9. Optimization

Model parameters are updated using backpropagation and an optimizer like Adam.

ALGORITHM:

1. Start
2. Input parallel text dataset
3. Convert sentences into numerical sequences
4. Pass input through encoder GRU
5. Obtain context vector
6. Feed context to decoder GRU
7. Generate output sequence word-by-word
8. Compute loss and update weights
9. Repeat for epochs
10. Translate new sentence
11. Stop

PROGRAM:

```
import torch
import torch.nn as nn
import torch.optim as optim


# English vocab
eng_vocab = {"I":0, "dont":1, "know":2, "you":3}
# Hindi vocab
hin_vocab = {"मैं":0, "तुम्हें":1, "नहीं":2, "जानता":3}

eng_vocab_size = len(eng_vocab)
hin_vocab_size = len(hin_vocab)

# reverse vocab for output
inv_hin_vocab = {v:k for k,v in hin_vocab.items()}


# "I dont know you"
input_seq = torch.tensor([[0,1,2,3]])   # shape (1,4)

# "मैं तुम्हें नहीं जानता"
target_seq = torch.tensor([[0,1,2,3]])  # shape (1,4)


class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.gru = nn.GRU(embed_size, hidden_size, batch_first=True)

    def forward(self, x):
        x = self.embedding(x)
        output, hidden = self.gru(x)
        return hidden
```

```
class Decoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.gru = nn.GRU(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden):
        x = self.embedding(x)
        output, hidden = self.gru(x, hidden)
        output = self.fc(output)
        return output, hidden


embed_size = 8
hidden_size = 16

encoder = Encoder(eng_vocab_size, embed_size, hidden_size)
decoder = Decoder(hin_vocab_size, embed_size, hidden_size)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(list(encoder.parameters()) + list(decoder.parameters()), lr=0.01)



for epoch in range(500):
        optimizer.zero_grad()
        hidden = encoder(input_seq)

    # Decoder input (start with first token)
        decoder_input = target_seq[:, :-1]   # remove last
        decoder_target = target_seq[:, 1:]   # remove first

        output, _ = decoder(decoder_input, hidden)

        loss = criterion(output.reshape(-1, hin_vocab_size), decoder_target.reshape(-1))

        loss.backward()
        optimizer.step()

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss.item()}")


hidden = encoder(input_seq)

decoder_input = torch.tensor([[0]])  # start with "मैं"
```

```
result = ["मैं",]

for _ in range(3):
    output, hidden = decoder(decoder_input, hidden)
    pred = output.argmax(2)
    result.append(inv_hin_vocab[pred.item()])
    decoder_input = pred

print("Translated Sentence:", " ".join(result))
```

SAMPLE OUTPUT:

Epoch 0, Loss: 1.450363039970398

Epoch 100, Loss: 0.002115861279889941

Epoch 200, Loss: 0.0009549643727950752

Epoch 300, Loss: 0.0005568296764977276

Epoch 400, Loss: 0.0003704732225742191

Translated Sentence: मैं तुम्हें नहीं जानता

RESULT:

Thus, the Python program to implement a machine translation using gated recurrent unit for sentence translation has been executed successfully.